



# Algorithmen und Datenstrukturen

Leon Johann Brettin



# Algorithmen und Datenstrukturen

11. Graphen – Algorithmen auf gewichteten Graphen



# Gewichtete Graphen

- ungerichtete gewichtete Graphen
  - Flugverbindungen mit Meilen oder Kosten
  - Straßennetze mit km
  - Rohrsysteme mit Durchfluss
  - bidirektionale Netzwerkverbindungen
- gerichtete gewichtete Graphen
  - Straßennetze mit Einbahnstraßen
  - Rohre mit Ventilen
  - Förderbänder



# Kürzeste Wege

- Graph  $G = (V, E, \gamma)$  mit einer Gewichtsfunktion  $\gamma: E \rightarrow \mathbb{N}$
- Weg oder auch Pfad (path) durch  $G$  ist Liste von aneinanderstoßenden Kanten:  
 $P = (v_1, v_2), (v_2, v_3), (v_3, v_4), \dots, (v_{n-1}, v_n)$
- Gewicht oder auch Länge eines Pfades: Aufsummierung der einzelnen Kantengewichte

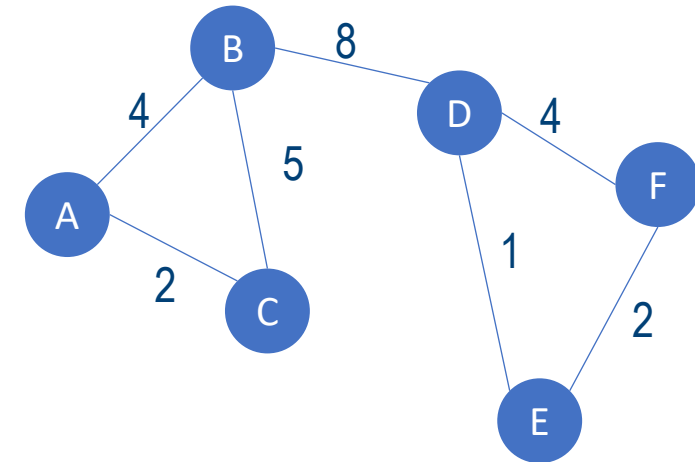
$$w(P) = \sum_{i=1}^{n-1} \gamma(v_i, v_{i+1})$$

- **Distanz** zweier Punkte  $d(u,v)$  ist **Gewicht des kürzesten Pfades** von  $u$  nach  $v$



# Minimaler Spannbaum: Kruskal

- Gegeben: gewichteter ungerichteter Graph
- minimaler Spannbaum  $B$  wird konstruiert  $\rightarrow$  minimale Summe der Kantengewichte
  - $B=(V,\emptyset)$
  - Füge die Kante mit dem kleinsten Gewicht dazu, falls sie zwei Komponenten von  $B$  verbindet.
  - Weiter, bis alle Kanten betrachtet.
- Greedy Algorithmus
  - Heuristik, die hier funktioniert (d.h. zum minimalen Spannbaum führt).



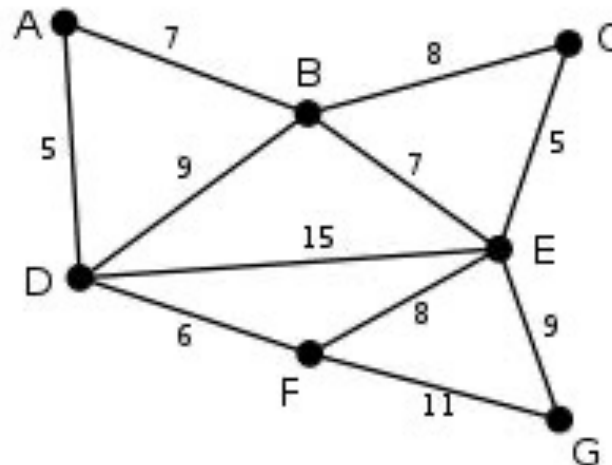


# Optimaler Spannbaum: Greedy

- Gegeben eine Teilmenge der Knoten / Kanten
  - und ein optimaler Spannbaum für diese Knoten
- Füge einen Knoten / Kante dazu
- und erweitere den Spannbaum, so dass er weiterhin optimal ist.
- Zwei Möglichkeiten:
  - Teilmenge ist und bleibt zusammenhängend (Prim)
  - Teilmenge kann unzusammenhängend sein (Wald)
    - charakterisiert durch Menge von Kanten (Kruskal)

# Algorithmus von Kruskal

- Joseph Kruskal, 1956 in „Proceedings of the American Mathematical Society“ veröffentlicht.
  - „Führe den folgenden Schritt so oft wie möglich aus: Wähle unter den noch nicht ausgewählten Kanten von  $G$  (dem Graphen) die kürzeste Kante, die mit den schon gewählten Kanten keinen Kreis bildet.“





# Formalisierung

```
algorithm Kruskal(V, E, w)
  Eingabe V: Knotenmenge, E: Kantenmenge, w: E → ℝ Gewicht
  E1 := ∅
  while (V, E1) nicht zusammenhängend do
    wähle die kleinste Kante e ∈ E \ E1,
      die zwei Komponenten in (V, E1) verbindet
    {oder: die keinen Kreis erzeugt}
    E1 := E1 ∪ {e}
  od
```





# Formalisierung

```
algorithm Kruskal(V, E, w)
  Eingabe V: Knotenmenge, E: Kantenmenge, w: E → ℝ Gewicht
  E1 := ∅
  while (V, E1) nicht zusammenhängend do
    wähle die kleinste Kante e ∈ E \ E1,
      die zwei Komponenten in (V, E1) verbindet
    {oder: die keinen Kreis erzeugt}
    E1 := E1 ∪ {e}
  od
```



# Formalisierung der Verfeinerung

```
algorithm Kruskal(V,E,w)
  E1 := ∅
  while (V,E1) nicht zusammenhängend do
    minKandidat := null
    foreach e ∈ E \ E1 do
      if KeinKreis(V,E1 ∪ {e}) then
        if (w(e) < w(minKandidat)) then
          minKandidat := e
        fi
      fi
    od
    E1 := E1 ∪ {minKandidat}
  od
```

Aufwand  
 $O(|V|^2 * |E|)$



# Formalisierung der Verfeinerung

```
algorithm Kruskal(V,E,w)
  E1 := ∅
  while (V,E1) nicht zusammenhängend do
    minKandidat := null
    foreach e ∈ E \ E1 do
      if KeinKreis(V,E1 ∪ {e}) then
        if (w(e) < w(minKandidat)) then
          minKandidat := e
        fi
      fi
    od
    E1 := E1 ∪ {minKandidat}
  od
```

Aufwand

$|V|$ -mal

$|E| - |E1|$ -mal

$O(|E1|) = O(|V|)$

Also:

$O(|V|^2 * |E|)$



# Suche nach kleinster Kante

- Kleinste Kante wird immer wieder benötigt.
  - Menge  $M$
  - `smallest()`
  - `remove(e)`
- Welche Datenstrukturen fallen Ihnen dazu ein?



# Suche nach kleinster Kante

- Kleinste Kante wird immer wieder benötigt.
  - Menge  $M$
  - `smallest()`
  - `remove(e)`
- Welche Datenstrukturen fallen Ihnen dazu ein?
  - Sortierte Liste
  - Priority Queue
    - als sortierter ausgeglichener Baum
  - Heap



# Verfeinerung Sortierte Kanten

```
algorithm Kruskal(V,E,w)
  E1 := ∅
  L := sort(E) // sortiert nach Gewichten.
  while (V,E1) nicht zusammenhängend do
    e := L.first();
    L.remove(e);
    while not KeinKreis(V,E1 ∪ {e}) do
      e := L.first();
      L.remove(e);
    od
    E1 := E1 ∪ {e}
  od
```

Aufwand  
 $O(|V| * |E| + |E| * \log(|E|))$



# Verfeinerung Sortierte Kanten

```
algorithm Kruskal(V,E,w)
  E1 := ∅
  L := sort(E) // sortiert nach Gewichten.
  while (V,E1) nicht zusammenhängend do
    e := L.first();
    L.remove(e);
    while not KeinKreis(V,E1 ∪ {e}) do
      e := L.first();
      L.remove(e);
    od
    E1 := E1 ∪ {e}
  od
```

Aufwand

$|E| \cdot \log(|E|)$

$|V|$ -mal insgesamt  
 $O(|E|)$ -mal

$O(|E1|) = O(|V|)$   
 $O(1)$

Also:  
 $O(|V| \cdot |E| + |E| \cdot \log(|E|))$



# Schrittweise Verfeinerung

- Teste, ob  $(V, E1 \cup \{e\})$  keinen Kreis enthält.

≡

- Teste, ob bei  $e=\{A,B\}$  A und B in verschiedenen Komponenten des Graphen  $(V, E1)$  liegen.
- $\text{KeinKreis}(V, E)$ 
  - Zum Beispiel: Tiefensuche in  $(V, E1 \cup \{e\})$ .





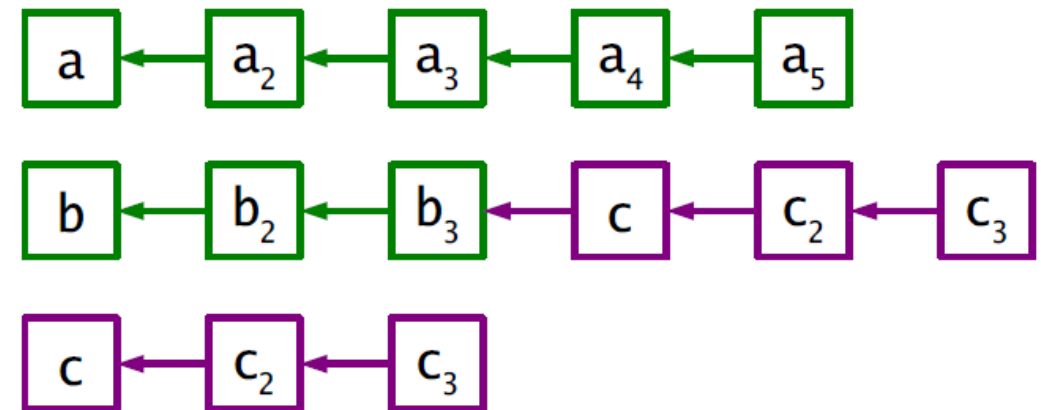
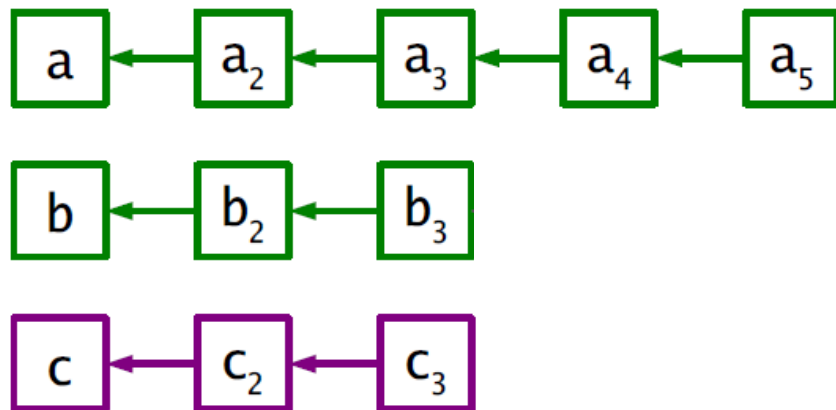
# Schrittweise Verfeinerung/Verbesserung

- $\text{KeinKreis}(V, E)$ 
  - Teste, ob bei  $e=\{A,B\}$  A und B in verschiedenen Komponenten des Graphen  $(V, E_1)$  liegen.
- Am besten: geeignete Datenstruktur mit Operationen
  - $\text{find}(A)$ : liefert einen Repräsentanten der Komponente, in der A liegt.
  - $\text{union}(R,S)$ : vereinigt die Komponenten, die durch R und S repräsentiert sind.
  - Union-Find Struktur
    - Benutzt Bäume oder Listen
    - Komplexität fast  $O(1)$  im Durchschnitt ist möglich.



# Union-Find: Verkettete Listen

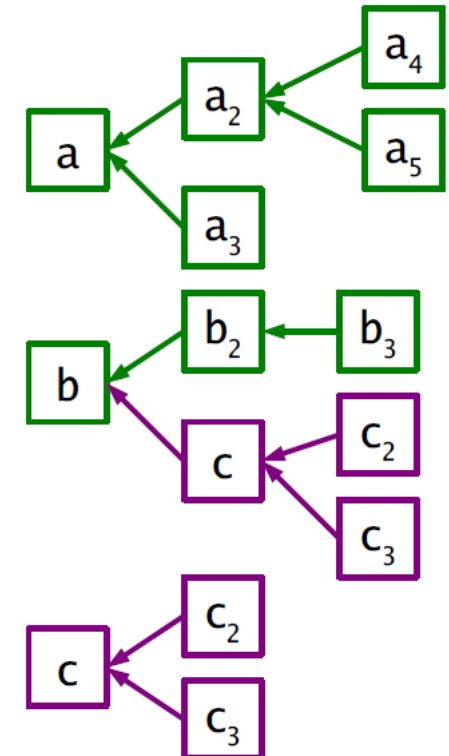
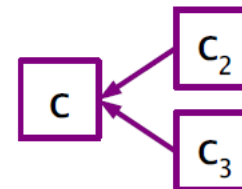
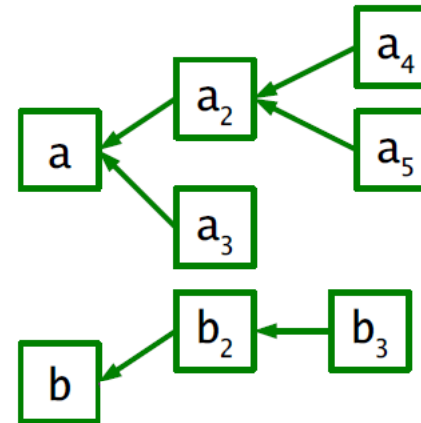
- Implementierung als verkettete Listen
  - find:  $O(n)$
  - union:  $O(1)$ .





# Union-Find: Bäume

- Implementierung als Bäume
  - find:  $O(h)$   
h: Höhe, maximal n,  
balanciert:  $\log(n)$
  - union:  $O(1)$

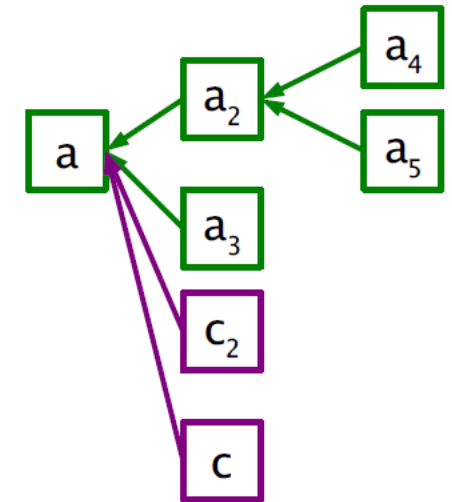
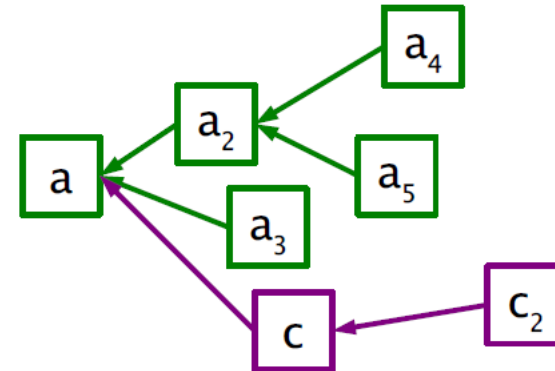




# Union-Find: balancierte Bäume mit Pfadkompression: fast $O(1)$

- Implementierung als Bäume
  - Union: kleineren Baum an größeren hängen
  - Find: Kind direkt an Wurzel hängen (Pfadkompression)!

```
function Find(x)
    if x.parent = x then
        return x // Wurzel
    else
        x.parent := Find(x.parent);
    return x.parent
fi
```





# Verfeinerung Union-Find

```
algorithm Kruskal(V, E, w)
    E1 := ∅
    L := sort(E);
    uf := UnionFind(V, ∅); // init
    while uf.countKomponents() > 1 do
        e := L.first();
        L.remove(e);
        while (a:=uf.find(e.A)) = (b:=uf.find(e.B)) do
            e := L.first();
            L.remove(e);
        od
        uf.union(a, b);
        E1.add(e);
    od
```

Aufwand  
 $O(|V| + |E| + |E| \cdot \log(|E|))$



# Verfeinerung Union-Find

```
algorithm Kruskal(V, E, w)
  E1 := ∅
  L := sort(E);
  uf := UnionFind(V, ∅); // init
  while uf.countKomponents() > 1 do
    e := L.first();
    L.remove(e);
    while (a:=uf.find(e.A)) = (b:=uf.find(e.B)) do
      e := L.first();
      L.remove(e);
    od
    uf.union(a, b);
    E1.add(e);
```

Aufwand

$|E| \cdot \log(|E|)$

$|V|$

$|V|$ -mal insgesamt

$O(|E|)$ -mal

$O(1)$

$O(1)$

Also:

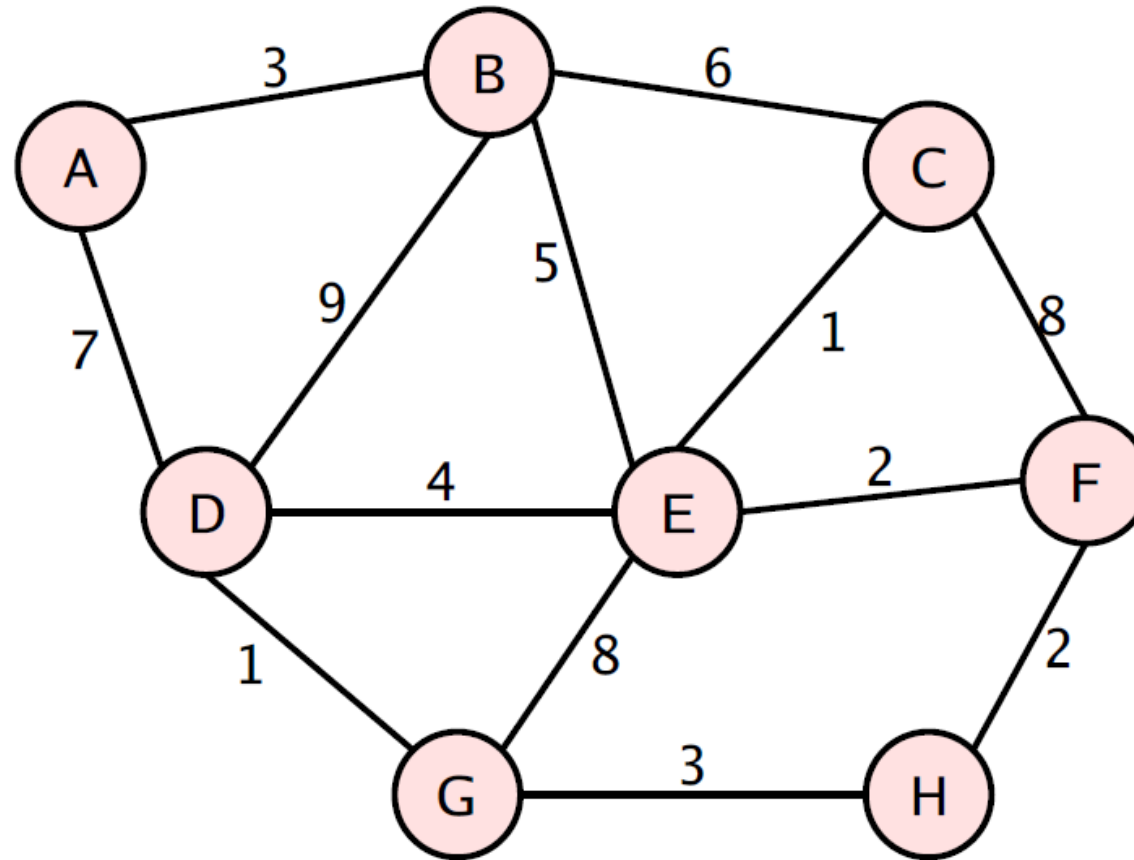
$O(|V| + |E| +$

$|E| \cdot \log(|E|))$

od

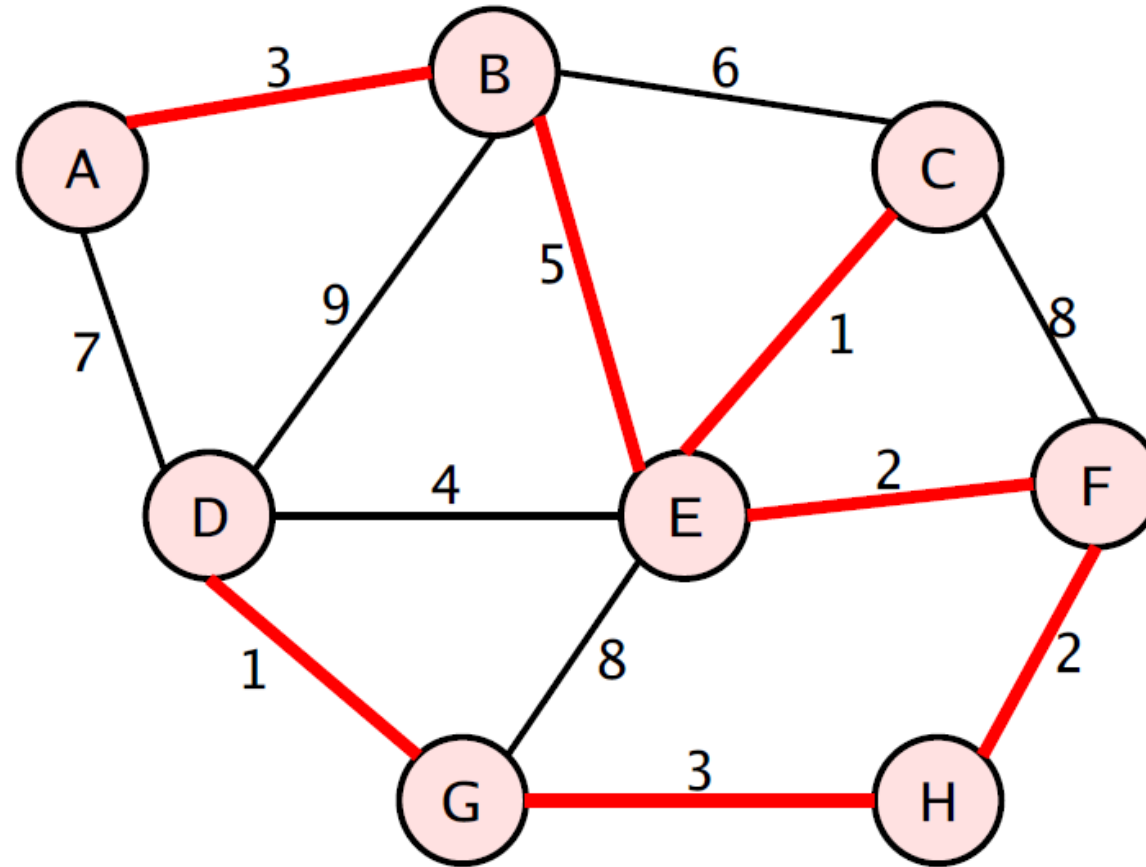


# Beispiel Kruskal-Algorithmus





# Beispiel Kruskal-Algorithmus







# Dijkstras Algorithmus

- Jetzt: Suche optimalen Pfad zu jedem Knoten, nicht optimalen Spannbaum!
- Dijkstra 1959
- iterative Erweiterung einer Menge von „billig“ erreichbaren Knoten
- Greedy-Algorithmus
  - ähnlich Breitensuche
  - nur für nicht-negative Gewichte
  - berechnet (iterativ verfeinernd) Distanzwerte  $D$
  - Prioritätswarteschlange zum Herauslesen des jeweils minimalen Elements

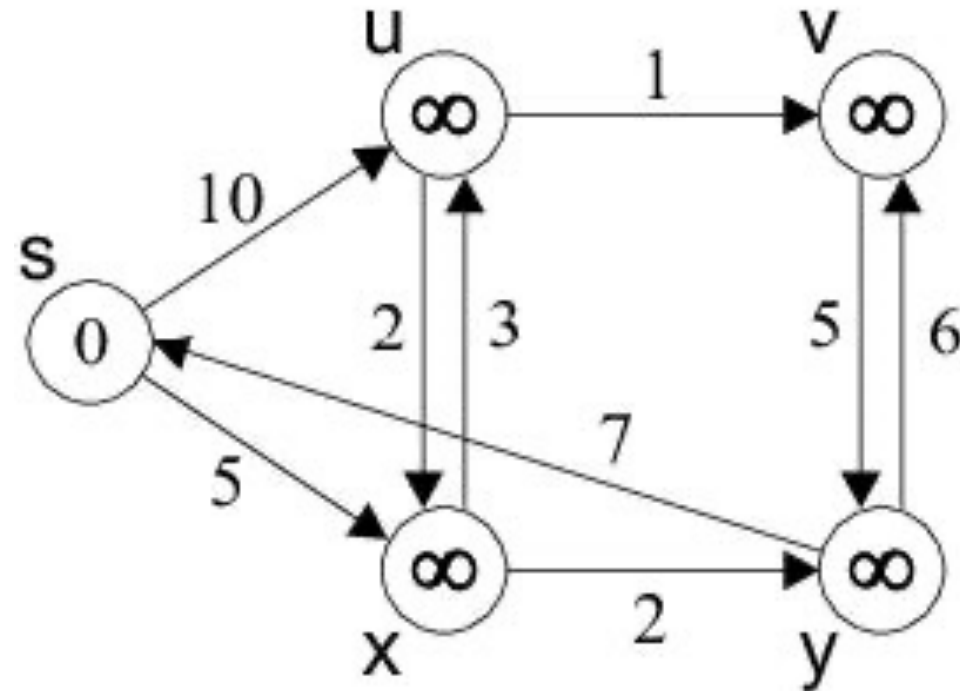


# Dijkstras Algorithmus

```
algorithm Dijkstra(G, s)
  Eingabe: ein Graph G mit Startknoten s
  foreach Knoten u aus V[G]\s do
    D[u] := unendlich
  od
  D[s] := 0;
  PriorityQueue Q := V;
  while ! Q.isEmpty do
    u := Q.extractMinimal();
    foreach v in („Zielknoten ausgehender Kanten“ von u) ∩ Q do
      if D[u] + gamma( (u, v) ) < D[v] then
        D[v] := D[u] + gamma( (u, v) );
        pi[v] := u; // Zeiger auf Vorgänger
        adjustiere Q an neuen Wert D[v]
      fi
    od
  od
```



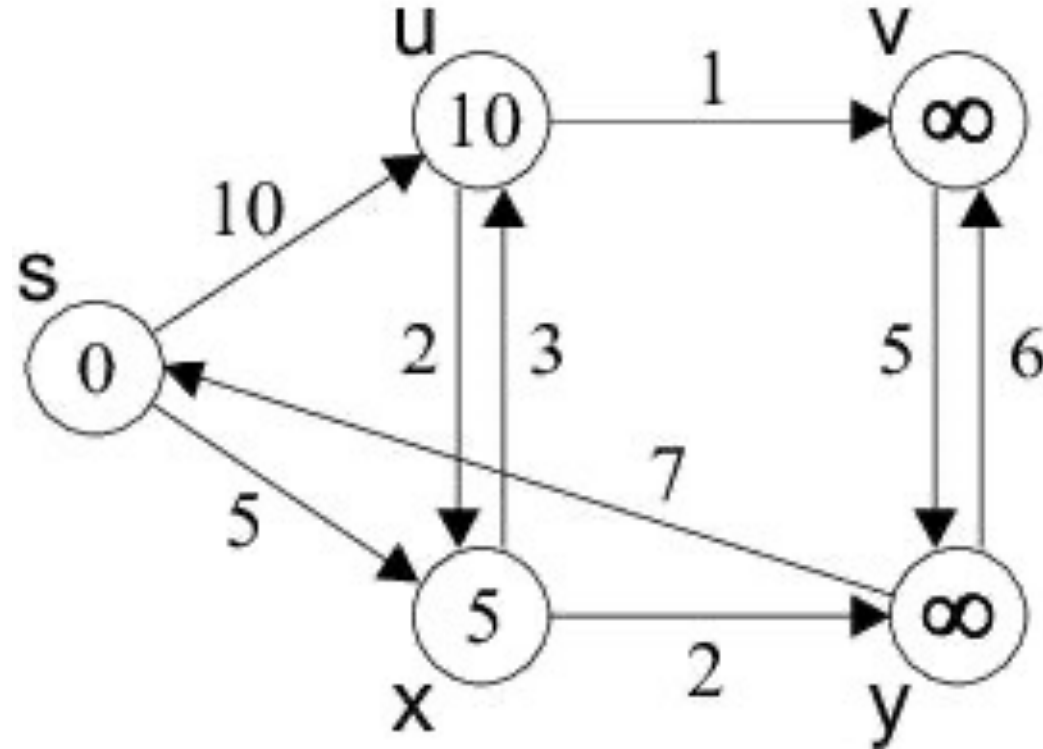
# Dijkstras Algorithmus: Initialisierung



$Q = \langle (s:0), (u: \infty), (v: \infty), (x: \infty), (y: \infty) \rangle$



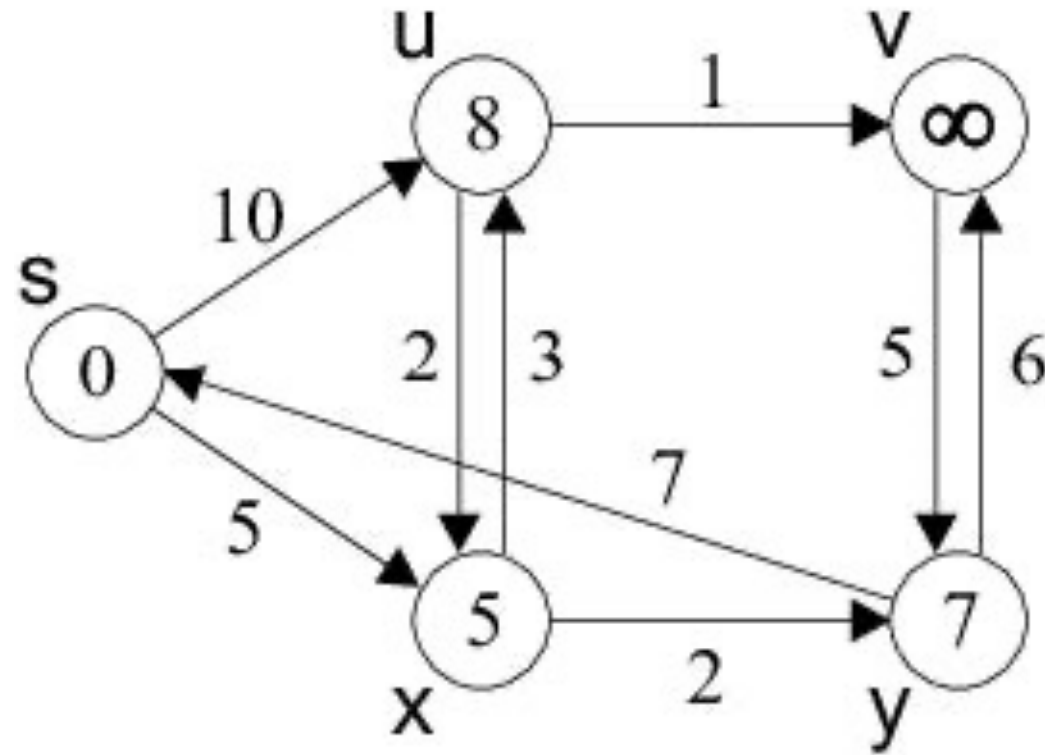
# Dijkstras Algorithmus: Schritt 1



$Q = \langle (x: 5), (u: 10), (v: \infty), (y: \infty) \rangle$



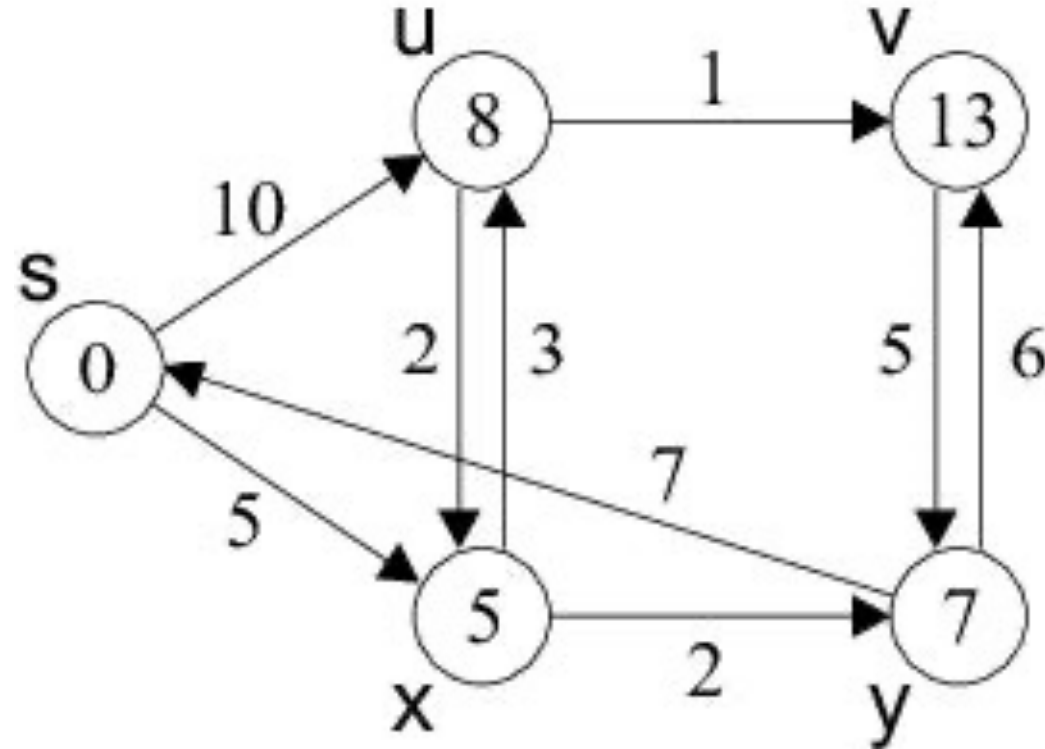
# Dijkstras Algorithmus: Schritt 2



$Q = \langle (y: 7), (u: 8), (v: \infty) \rangle$



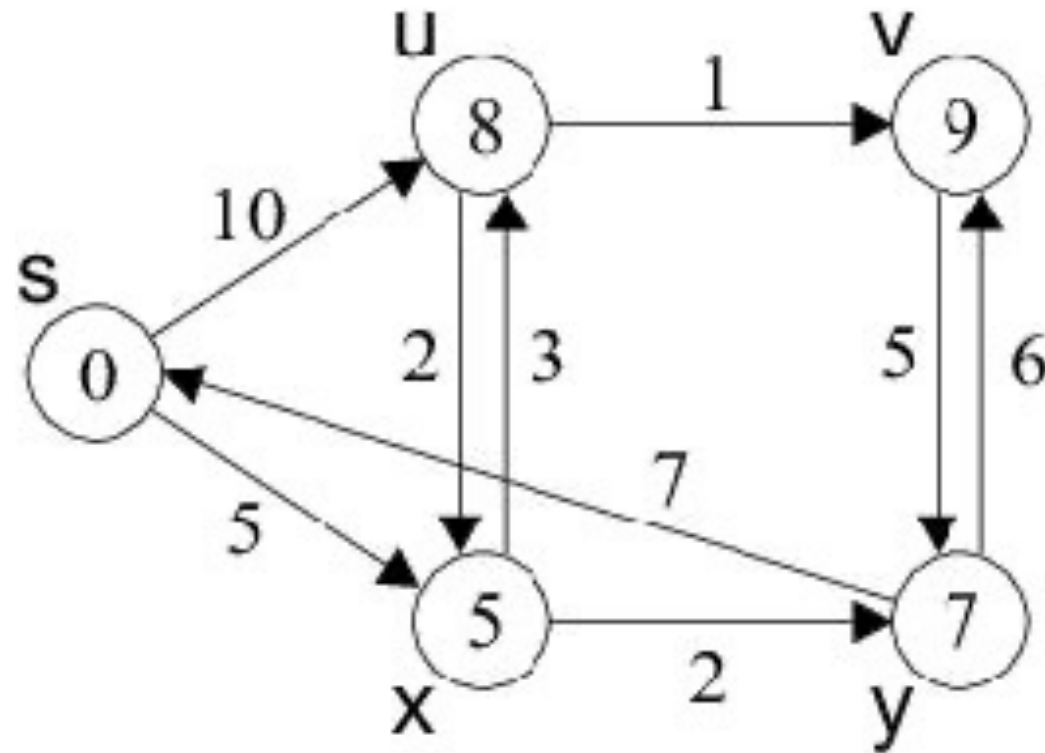
# Dijkstras Algorithmus: Schritt 3



$Q = < (u: 8), (v: 13) >$



# Dijkstras Algorithmus: Schritt 4



$Q = \langle (v: 9) \rangle$



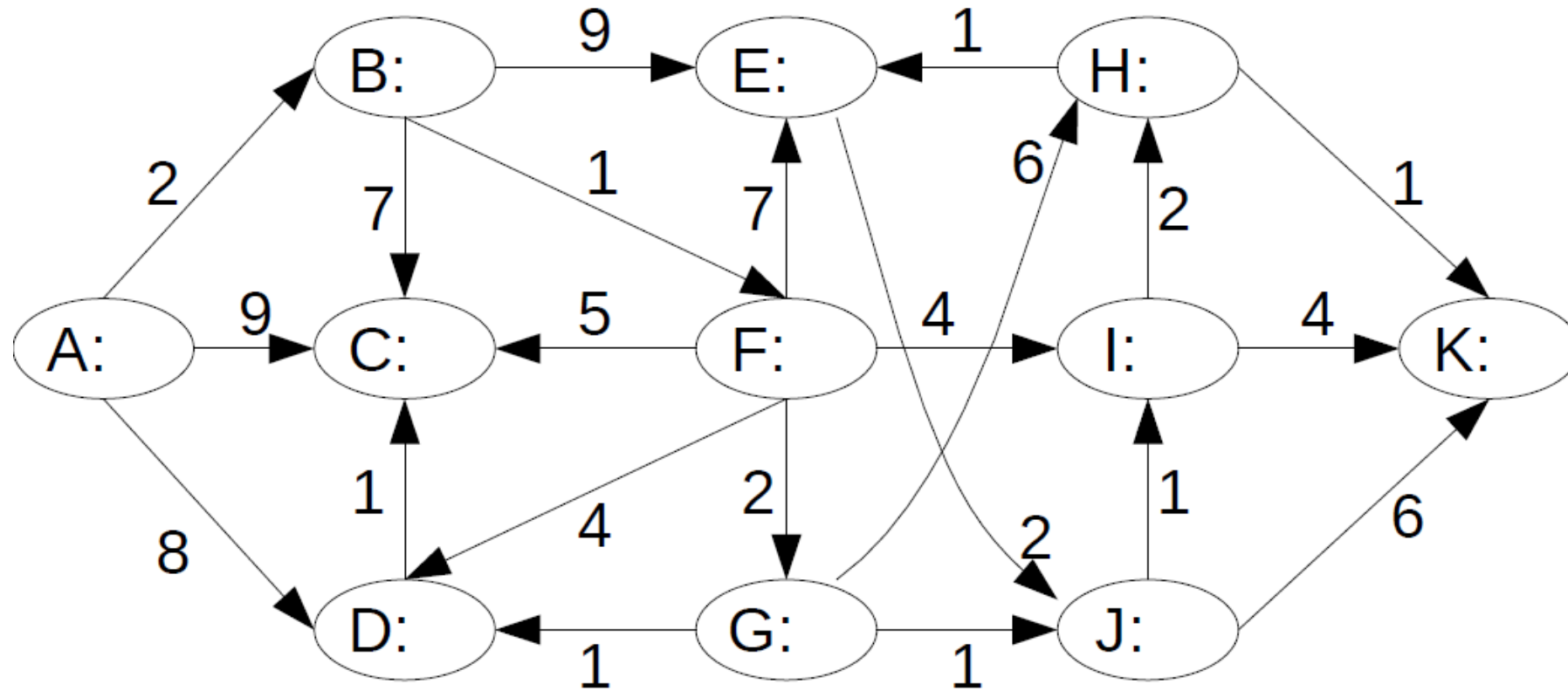
# Wird das Optimum berechnet?

- Annahme: vorherige Iterationsschritte korrekt (Induktionsbeweis)
- Iterationsschritt: jeweils die „billigste“ Verbindung zu einem noch nicht bearbeiteten Knoten hinzunehmen
  - Da die bisher bearbeiteten Knoten den korrekten Distanzwert haben, ist der neue Distanzwert durch den „billigsten Pfad“ bestimmt, der aus dem bisher bearbeiteten Teilgraphen um genau eine Kante hinausgeht.
  - Jeder Pfad zum Zielknoten dieses Pfades, der um mehr als eine Kante aus dem bearbeiteten Bereich hinausgeht, ist teurer als die gewählte, da Kosten mit zusätzlich hinzugenommenen Kanten nicht sinken können (nichtnegative Gewichte)!



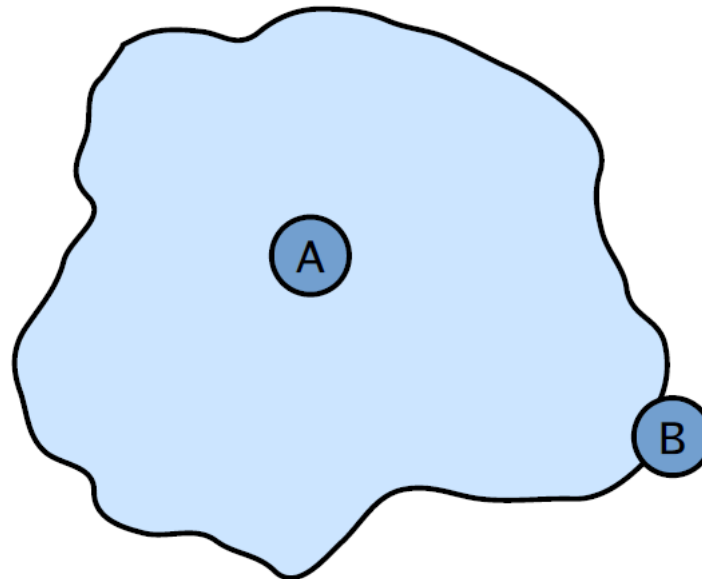


# Üben



# Suche nach dem kürzesten Weg $A \rightarrow B$

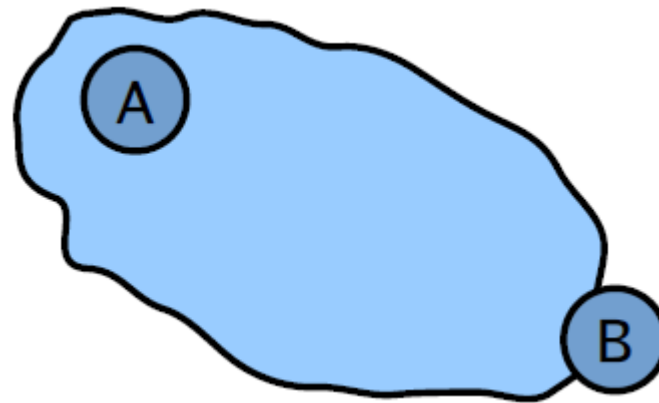
- Dijkstra sucht den kürzesten Weg zu allen Zielen.
- Können wir das besser?
  - Suche abbrechen, wenn Zielknoten B erreicht wurde.





# Suche nach dem kürzesten Weg $A \rightarrow B$

- Suche abbrechen, wenn Zielknoten B erreicht wurde.
- Suche in die richtige Richtung präferieren.
- A\*-Algorithmus





# A\*-Algorithmus: $h(u)$

- $g(u)$ : bisher berechnete Kosten vom Startpunkt zu  $u$ .
- $h(u)$ : geschätzte (minimale) Kosten von  $u$  zum Ziel.
  - $h$  für Heuristik
  - muss leicht zu berechnen sein
  - Zulässigkeit  $h(u) \leq d(u)$  : echte Kosten von  $u$  zum Ziel sind höher als geschätzte Kosten, Kosten werden nicht überschätzt
  - Schärfer: Konsistenz (Monotonie, Dreiecksungleichung):  $h(u) \leq h(v) + \gamma(u,v)$
  - Beispiel: Entfernung in Luftlinie
- $f(u) = g(u) + h(u)$ 
  - Untergrenze für Gesamtkosten von Wegen über  $u$



# A\*-Algorithmus: Datenstrukturen

- OPEN: Liste aller Knoten, zu denen ein Weg gefunden wurde, mit f-Werten.
  - Operationen: create, contains, enqueue, extractMinimal, isEmpty  
→ Priority Queue
- CLOSED: Knoten, zu denen der kürzeste Weg bekannt ist
- Alle anderen noch unbekannt.
- Jeder Knoten:
  - $h(u)$  Konstante
  - $\pi(u)$  Zeiger zum aktuellen Vorgänger
  - $g(u)$  wird berechnet



# A\*-Algorithmus

- Startknoten in die OPEN Liste eintragen,  $g(u)=0$
- *Solange* OPEN nicht leer und Ziel nicht erreicht *do*
  - Knoten mit kleinstem f-Wert in OPEN auswählen: u
  - u = Ziel: fertig
  - Alle Nachbarn v von u betrachten:
    - v schon in CLOSED: ignorieren
    - f-Wert bei Weg über u und v berechnen
      - gibt es schon besser: ignorieren
      - sonst: v in OPEN Liste einfügen
      - f- und g-Werte aktualisieren
  - u aus OPEN in CLOSED transferieren



# A\*-Algorithmus

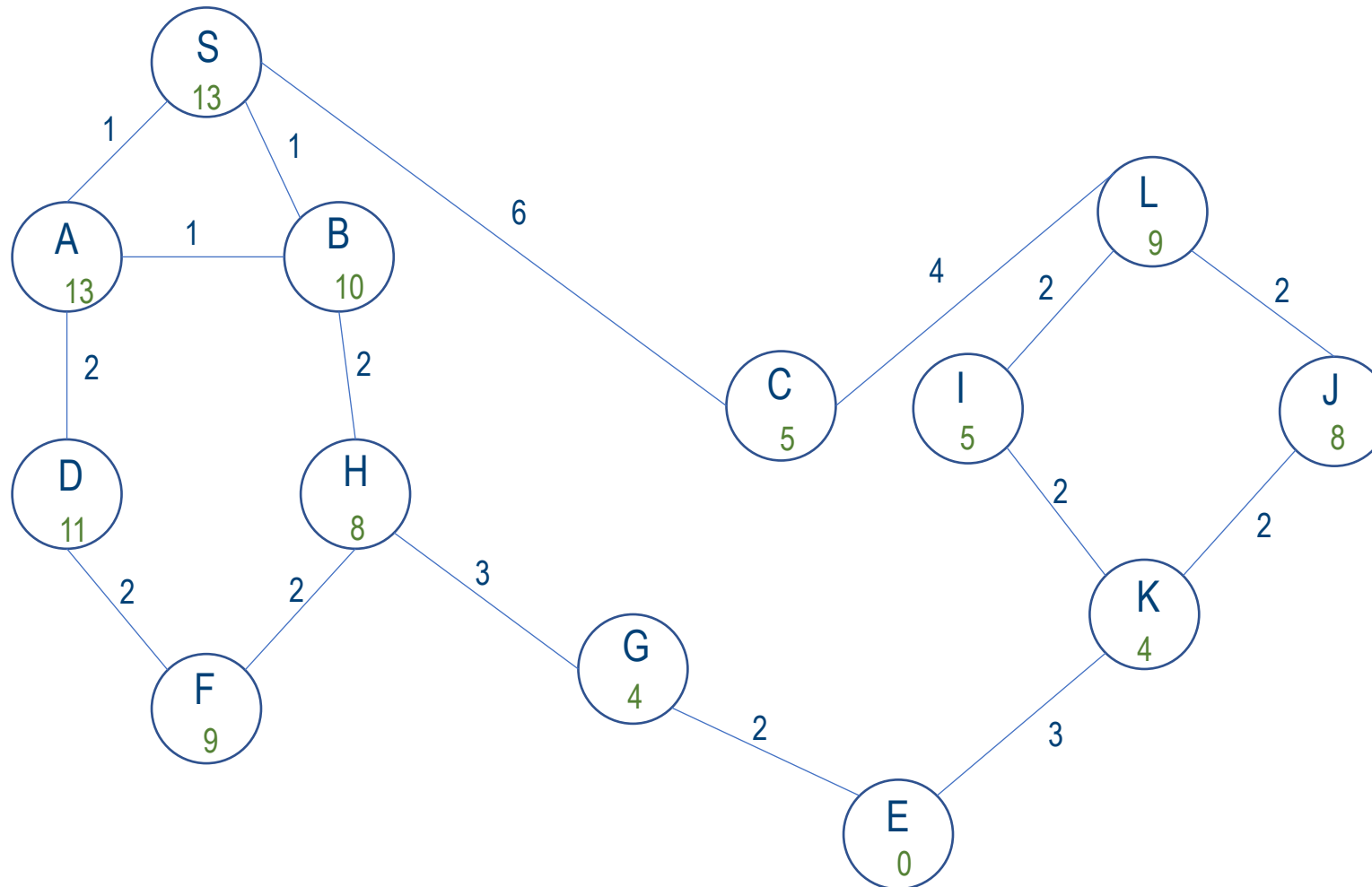
```
algorithm A*-Suche (G, s, z)
  Eingabe: Graph G mit Start  $s \in G[V]$  und Ziel  $z \in G[V]$ 

  PriorityQueue OPEN := {}, Liste CLOSED := {}
  OPEN := enqueue(OPEN, s);
  while  $\neg$ isEmpty(OPEN) do
    n := extractMinimal(OPEN);
    if  $n = z$  then return WegGefunden; fi;
    expand(n);
    CLOSED := enqueue(CLOSED, n);
  od;
  return WegNichtGefunden
```

```
procedure expand(Knoten  $u \in G[V]$ )
begin
  foreach  $v \in \text{ZielknotenAusgehenderKanten}(u) - \text{CLOSED}$  do
     $tf := g(u) + \gamma(u, v) + h(v)$ 
    if  $v \in \text{OPEN} \wedge tf > f(v)$  then continue; fi;
     $v.\text{pred} := u$ ;
    Aktualisiere f- und g-Kosten von v;
    if  $v \notin \text{OPEN}$  then
      OPEN := enqueue(OPEN, v);
    fi
  od
end
```



# A\*-Algorithmus Beispiel





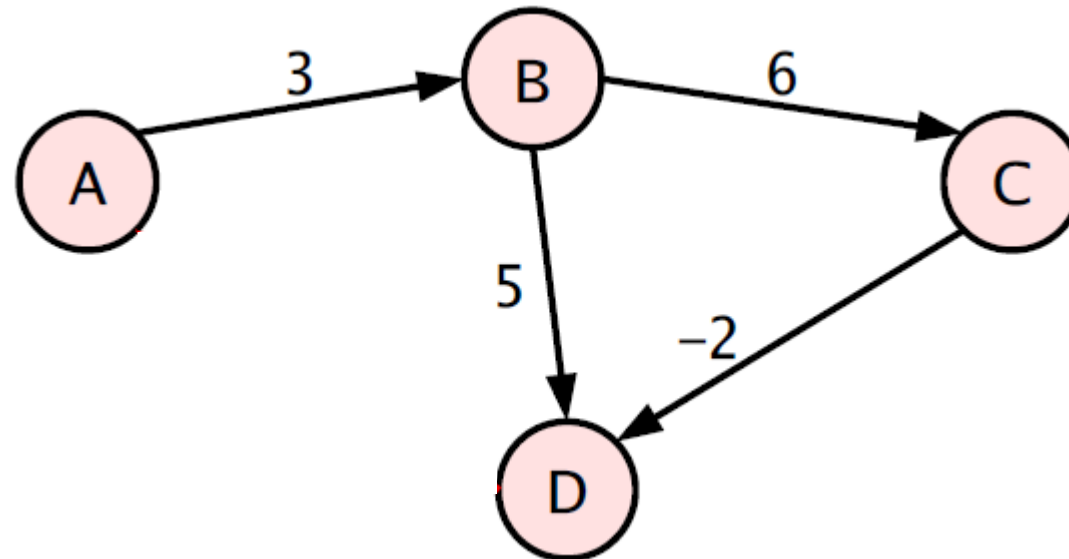


# A\*: weitere Verbesserung

- möglichst gute Abschätzung des Minimums
- Von Start und Ziel aus gleichzeitig suchen
  - Wenn die Suchmengen (OPEN) sich schneiden, ist der optimale Weg gefunden.
- Java-Code: siehe Buch.
  - verwendet PriorityQueue
  - kann nicht Prioritäten in der Datenstruktur ändern
  - Lösung: Element entfernen, Wert ändern, neu einfügen.

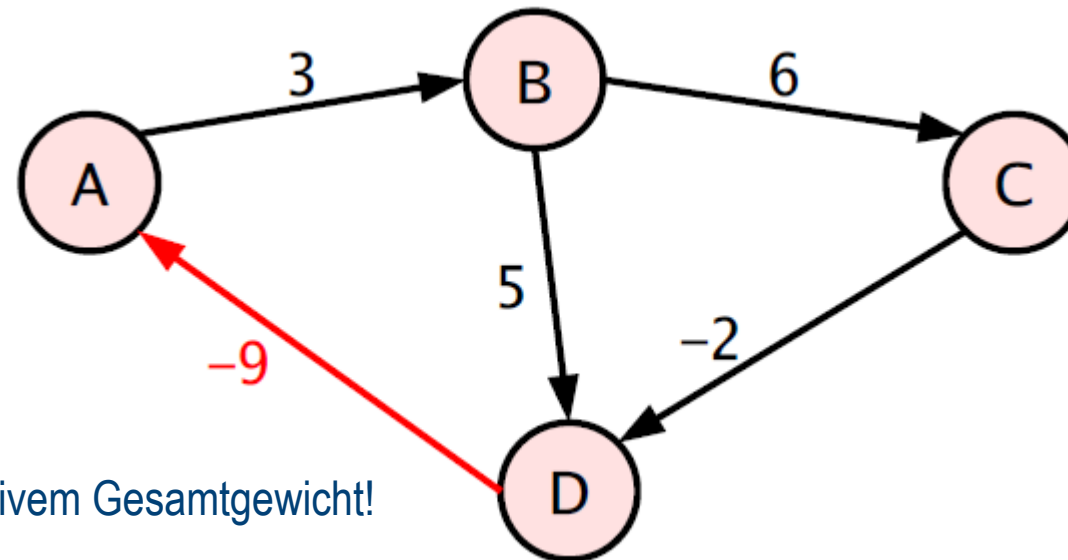
# Negative Kantengewichte

- Dijkstra-Algorithmus funktioniert nur, wenn  $\gamma(u,v) > 0$ .
- Denn sonst könnten negative Kantengewichte einen „ungünstigen“ Weg ausgleichen.



# Negative Kantengewichte

- Dijkstra-Algorithmus funktioniert nur, wenn  $\gamma(u,v) > 0$ .
- Denn sonst könnten negative Kantengewichte einen „ungünstigen“ Weg ausgleichen.



Keine Kreise mit negativem Gesamtgewicht!

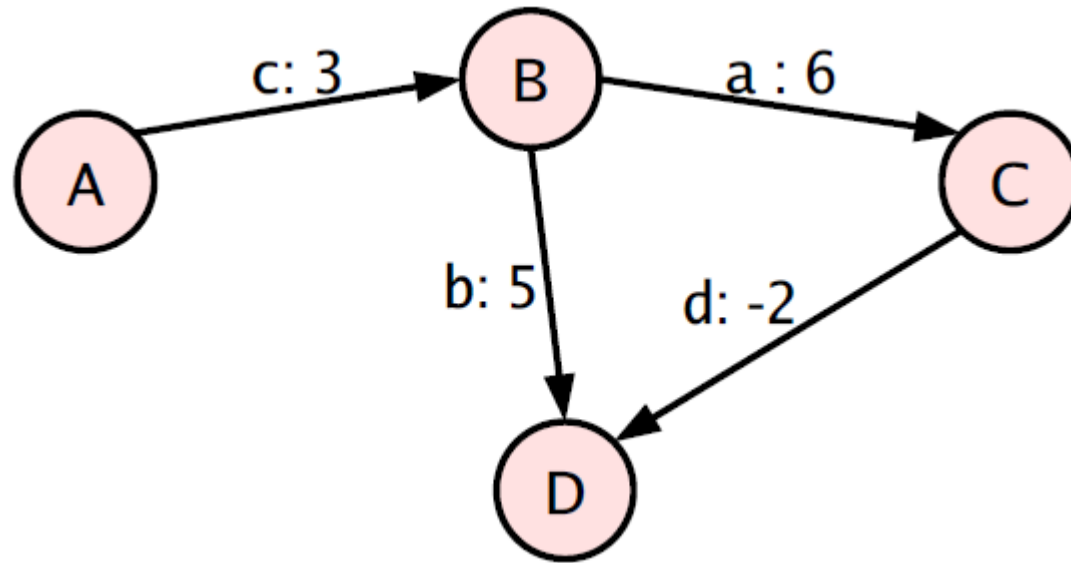


# Bellman-Ford-Algorithmus

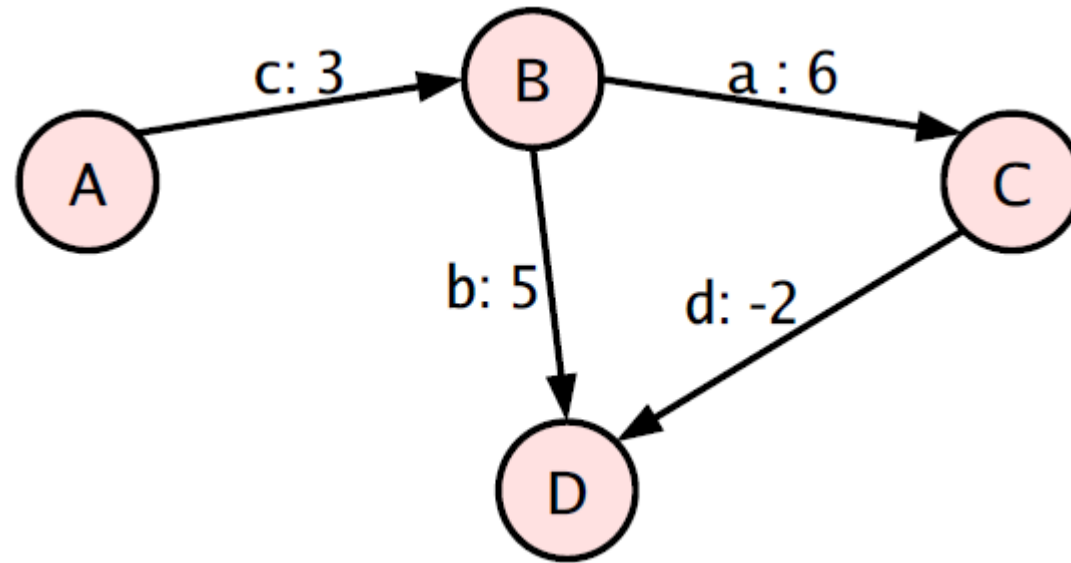
```
algorithm BellmanFord(G, s)
  Eingabe: ein Graph G mit Startknoten s
  foreach Knoten u aus V[G]\s do
    D[u] := unendlich
  od
  D[s] := 0;
  for i := 1 to |E| do // oder: solange sich etwas ändert
    foreach (u,v) aus E do
      if D[u] + gamma( (u, v) ) < D[v] then
        D[v] := D[u] + gamma( (u, v) );
      fi
    od
  od
```



# Bellman-Ford Beispiel



# Bellman-Ford Beispiel

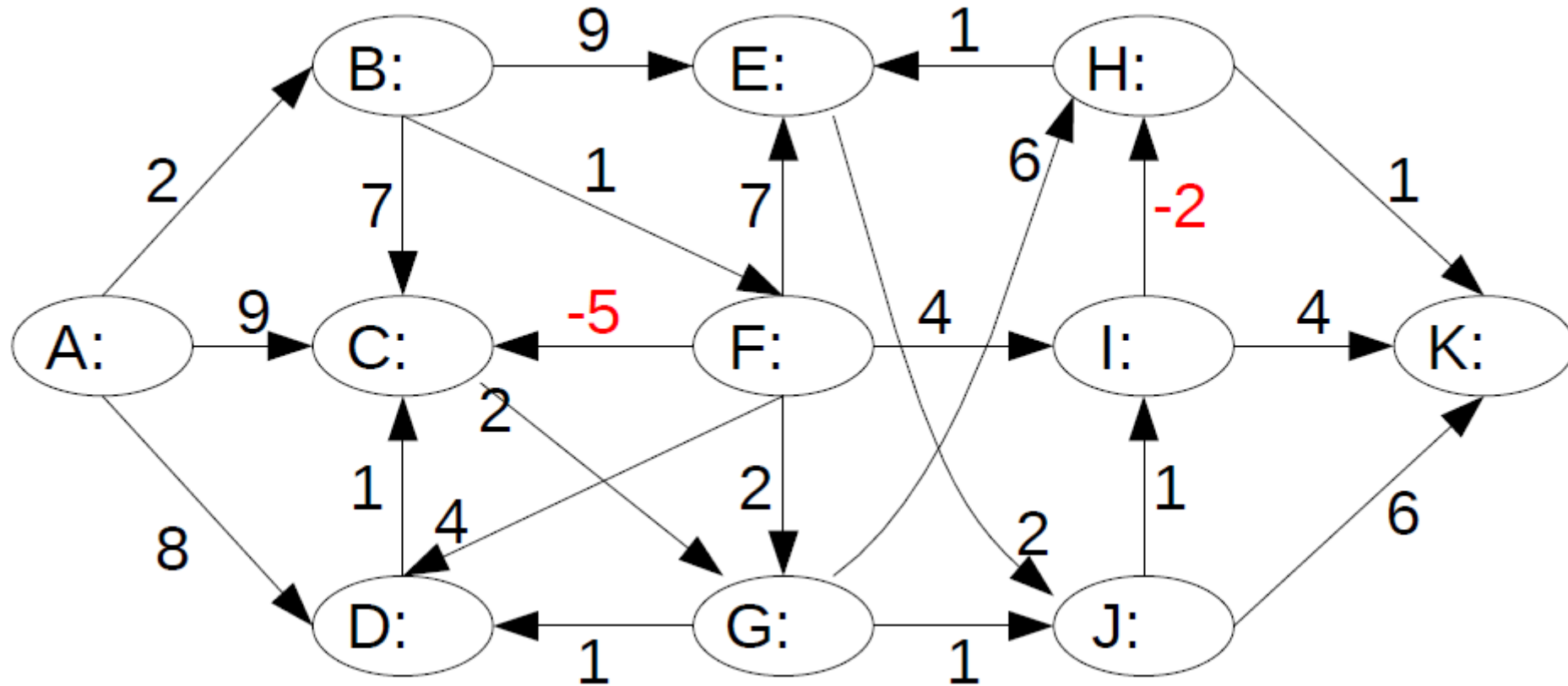


| Iteration | 0        | 1                    | 2   | 3 |
|-----------|----------|----------------------|-----|---|
| A         | 0        | 0                    | 0   | 0 |
| B         | $\infty$ | 3                    | 3   | 3 |
| C         | $\infty$ | $\infty+6$           | 9   | 9 |
| D         | $\infty$ | $\infty+5, \infty-2$ | 8,7 | 7 |

Besuche Kanten in der Reihenfolge  
a,b,c,d



# Üben





# Zusammenfassung

- Gewichtete Graphen (Netzwerke)
  - Knoten, Kanten, Gewichte
- Kleinster Spannbaum
  - Algorithmus von Kruskal (oder Prim)
  - greedy, verwendet Union-Find Datenstruktur
- Kürzeste Wege
  - Algorithmus von Dijkstra, A\*-Algorithmus
  - verwendet Priority Queue
  - $O((|V| + |E|) \log |V|)$
- Bellman-Ford
  - auch für negative Kantengewichte
  - $O(|E|^2) = O(|V|^4)$  (im schlimmsten Fall)





# Kruskal Üben

